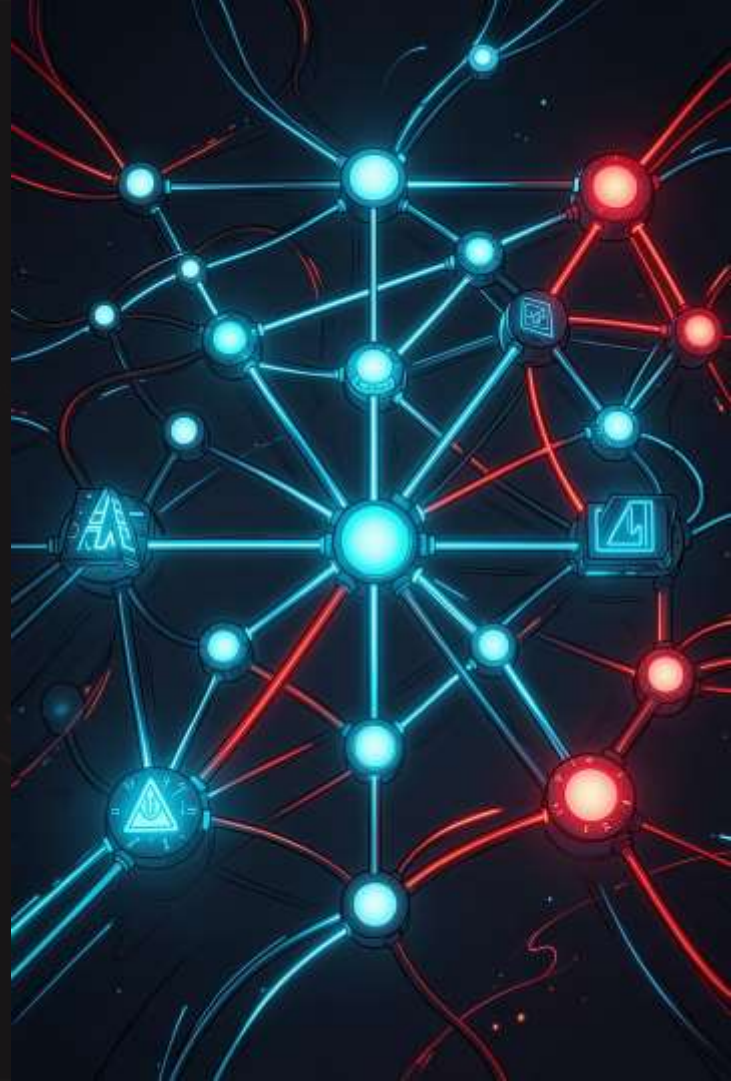
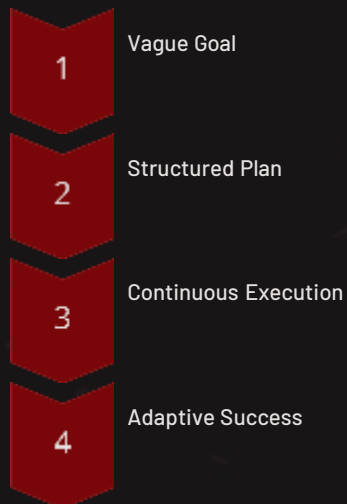


PathLoom

Adaptive Workflow Intelligence Platform

PathLoom transforms vague goals into structured execution workflows through AI-assisted planning, dependency reasoning, and adaptive replanning — continuously supporting execution as delays, blockers, and changing priorities emerge.



The Execution Gap

People set ambitious goals — build a startup, launch a SaaS product, learn ML — but most fail not from lack of ambition, but lack of structured, adaptive execution support.

To-Do Apps

Static task lists with no dependency awareness

AI Chatbots

One-time responses, zero execution follow-through

Project Tools

Require heavy manual setup before value is realized

Calendars

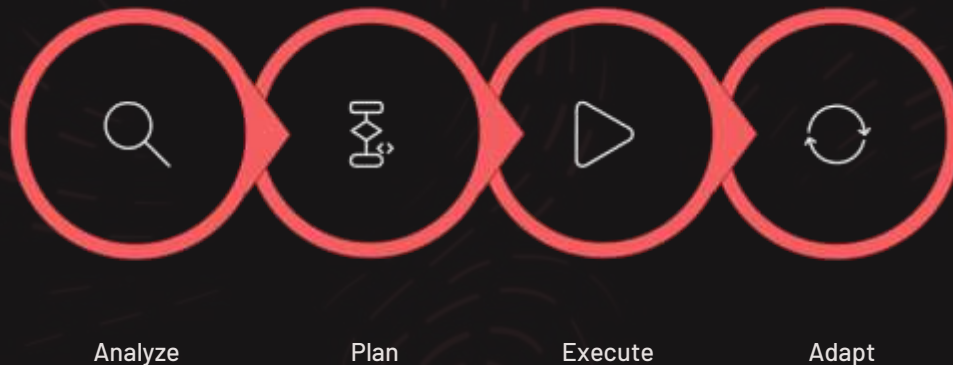
Track time, not meaningful execution progress

Key Insight: Most systems help users create plans. Very few help users **sustain execution**.



How PathLoom Works

From a single vague goal to a fully adaptive, continuously monitored execution workflow — automatically.



PathLoom's end-to-end pipeline converts a raw user goal into a living workflow that evolves in real time — not a static plan that becomes obsolete on day one.

Example Input

"Build an AI SaaS MVP in 30 Days"

PathLoom Output

- Structured milestones & tasks
- Dependency graph
- Priority-ranked work queue
- Live progress dashboard
- Adaptive replanning recommendations

Why Agentic AI?

Execution is not a one-time event — it's a continuously evolving process. Traditional LLM calls produce a static output and stop. PathLoom's agentic loop observes, reasons, acts, and adapts.

Traditional AI

01

User prompts

02

LLM responds

03

Tasks returned

04

End — no follow-through

PathLoom Agentic Loop



 Observe

 Analyze

 Plan

 Execute

 Monitor

 Adapt

Goal Analysis Agent

The first agent in the pipeline converts an ambiguous user goal into structured planning context — determining goal type, complexity, risks, and required workstreams before any workflow is generated.

Inputs

- Goal description
- Deadline & available hours
- Skill level & constraints



Example: "Build AI SaaS MVP in 30 Days"

Structured Output

```
{
  "goal_type": "Product Development",
  "complexity": "Medium",
  "risk_level": "Medium",
  "workstreams": [
    "Research",
    "Backend",
    "Frontend",
    "Testing",
    "Deployment"
  ]
}
```

This structured context anchors all downstream agents, ensuring consistent, goal-aware planning.

Task Decomposition & Dependency Agent

Two specialized agents work in tandem: one breaks the goal into milestones, tasks, and subtasks with effort estimates; the other maps execution relationships to surface the critical path.



Testing cannot begin before Backend completion. Deployment cannot begin before Testing passes. The dependency graph identifies the critical execution path and surfaces blocking risks before they become problems.

Milestones

Phased goal breakdown

Tasks & Subtasks

Granular action items

Effort Estimates

Time & complexity scoring

Dependency Graph

Critical path detection

Priority Agent

Not all tasks contribute equally toward goal completion. The Priority Agent scores every task using a weighted formula, ensuring users always focus on the highest-impact work first.

Priority Formula

$$\text{Score} = 0.35 \times \text{Dependency} + 0.30 \times \text{Deadline} + 0.20 \times \text{Goal Impact} + 0.15 \times \text{Effort}$$

Example: Backend API

Factor	Score
Dependency Impact	10
Deadline Urgency	8
Goal Impact	9
Effort Efficiency	6

Final Score: 8.55 → **High Priority**

Priority Tiers

● Critical	🔍 High
🔍 Medium	🔍 Low



Users always know what to work on next — no decision fatigue, no wasted effort.



SLIDE 8

Progress Tracking Agent

Most productivity systems stop after generating plans. **PathLoom actively monitors execution** – tracking completion, detecting delays, and measuring workflow health in real time.

What the Progress Agent Does

Responsibilities

- Track task completion rates
- Detect delays and blocked tasks
- Measure milestone progress
- Calculate overall workflow health

Inputs

- Task status updates
- Completion percentages
- User activity signals
- Deadline information

Outputs

✓ On Track

⚠️ At Risk

🛑 Blocked

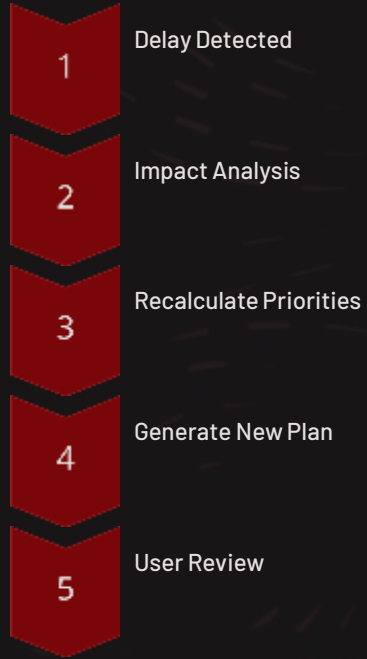
?? Delayed



Example: Backend API ✓ Completed ·
Authentication 🔄 In Progress · Testing 🛑 Blocked
→ Workflow Health: At Risk

Adaptive Replanning Agent

Execution rarely follows the original plan. When disruptions occur — delays, scope changes, resource constraints, or dependency failures — PathLoom recalculates and regenerates the plan automatically.

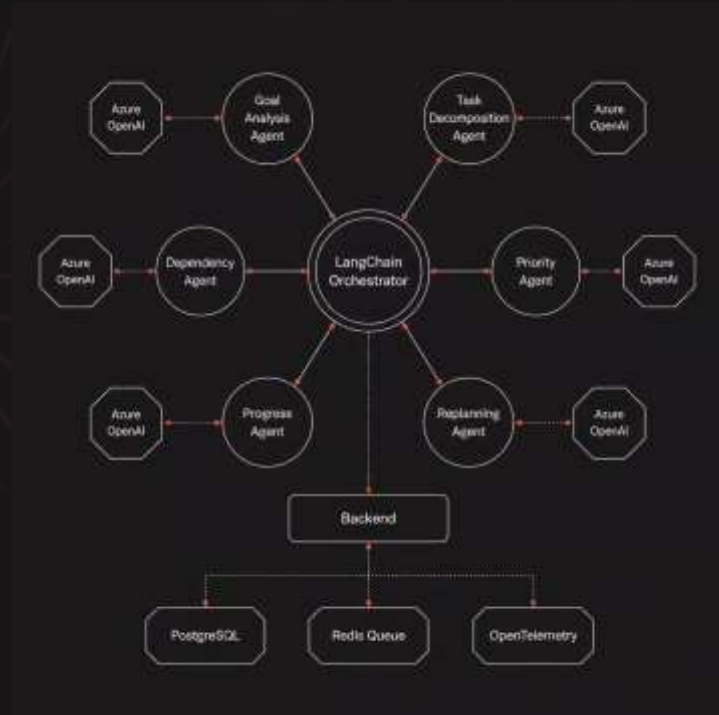


User options: Accept · Modify · Reject — keeping humans in control of execution continuity.

Complete System Architecture

PathLoom's architecture is organized into distinct layers, each with a clear responsibility – from user-facing interaction through AI reasoning to persistent storage and monitoring.

Layer	Responsibility
Frontend	User Interaction
Backend	API Management
LangChain	Agent Orchestration
Azure OpenAI	Reasoning Engine
PostgreSQL	Workflow Storage
Redis	Async Jobs
OpenTelemetry	Monitoring



Agent Orchestration Architecture

A single prompt cannot reliably handle goal analysis, task generation, dependency detection, prioritization, and replanning. **LangChain separates these responsibilities** into modular, testable agents.



This sequential pipeline ensures each agent receives structured input from the previous stage, improving accuracy and explainability at every step.

Modular Design

Each agent is independently testable

Better Explainability

Clear reasoning at every stage

Improved Reliability

Failures are isolated and recoverable

Scalable

New agents added without rewrites

Azure OpenAI: Understanding & Task Generation

Step 1 – Goal Understanding

The Goal Analysis Agent interprets the user's intent, classifying the goal type and complexity before any planning begins.



Input: *"Build AI SaaS MVP in 30 Days"*

Output: { "goal_type": "Product Development",
"complexity": "Medium" }

Step 2 – Task Generation

The Task Decomposition Agent breaks the goal into actionable workstreams, ordered by logical execution sequence.

01

Research

03

Backend

05

Testing

02

Architecture

04

Frontend

06

Deployment

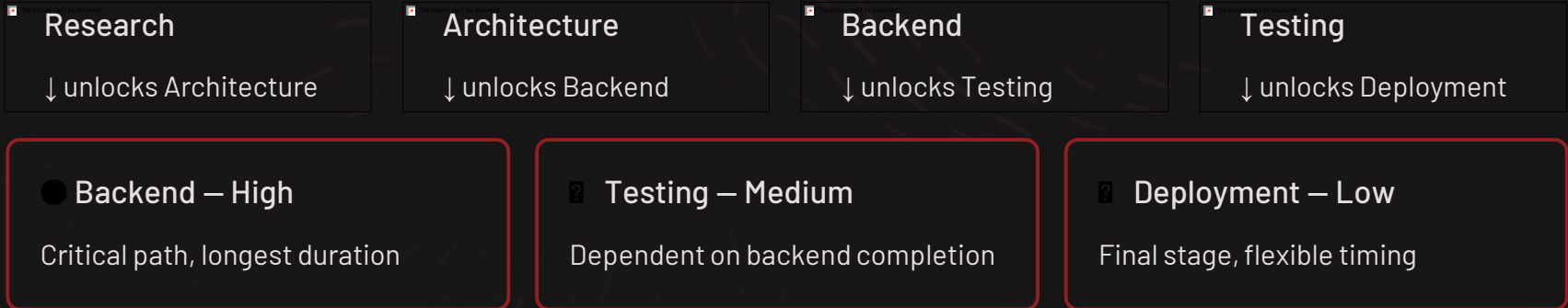
Dependency Detection & Priority Assignment

Step 3 — Dependency Detection

The Dependency Agent maps task relationships, ensuring critical-path ordering is preserved throughout the workflow.

Step 4 — Priority Assignment

The Priority Agent assigns urgency levels based on goal type, dependencies, and deadline constraints.



Replanning & The Case for Multiple Prompts

Step 5 – Replanning

When the Progress Agent detects a disruption, the Replanning Agent recalculates affected tasks and generates an updated workflow for user review.



Input: *"Backend delayed by 5 days"*

Output: Updated workflow with revised milestone dates and adjusted task order.

Why Multiple Prompts?

Splitting AI reasoning across focused prompts outperforms a single monolithic prompt on every dimension.



Accuracy

Each prompt has a focused, well-defined objective



Explainability

Every decision is traceable to a specific agent



Maintainability

Individual prompts can be tuned without side effects

PathLoom: From Static Plans to Living Workflows

PathLoom replaces rigid, one-time planning with an **intelligent, self-correcting execution system** — continuously monitoring, adapting, and guiding teams toward their goals.

Understand Goal Analysis Agent	Decompose Task & Dependency Agents	Prioritize Priority Agent
Monitor Progress Agent	Adapt Replanning Agent	

Agent Communication & Data Flow

Agents communicate through a shared data model, ensuring consistency and efficient information exchange. The LangChain orchestrator manages the flow, passing structured outputs from one agent as inputs to the next.

Key Communication Principles

- **Standardized Data Formats:** All agent inputs and outputs adhere to a predefined JSON schema.
- **Event-Driven Architecture:** Agents react to changes in the workflow state, triggering subsequent steps.
- **Orchestrated Flow:** LangChain ensures agents are called in the correct sequence, preventing race conditions or invalid states.
- **Auditable Trails:** Every agent interaction and data transformation is logged for transparency and debugging.



SLIDE 13

Database Architecture

PathLoom's data model is built around a **goal-centric hierarchy** — from high-level objectives down to individual tasks, dependencies, and AI-generated audit trails. Every entity is traceable, auditable, and interconnected.

Core Data Model

Seven interconnected tables form the backbone of PathLoom's persistence layer. **Goals** sit at the center, owning milestones, tasks, and generating a full audit trail of AI decisions and replanning events.



The picture can't be displayed. Users Identity & profile data	The picture can't be displayed. Goals High-level objectives	The picture can't be displayed. Milestones Major checkpoints	The picture can't be displayed. Tasks Actionable work items
The picture can't be displayed. Dependencies Execution relationships	The picture can't be displayed. ProgressUpdates Workflow progress snapshots	The picture can't be displayed. WorkflowHistory Replanning event log	The picture can't be displayed. AgentEvents AI decision audit trail

Goal & Task APIs

Goal APIs

Manage the lifecycle of a goal from creation through activation and AI-driven analysis.

POST `/goals`

Create a new goal

GET `/goals/{id}`

Retrieve goal details

POST
`/goals/{id}/analyze`

Trigger AI analysis

POST
`/goals/{id}/activate`

Begin execution

GET `/tasks`

List all tasks

PATCH `/tasks/{id}`

Update task fields

Task APIs

Control task state transitions — completion, blocking, and updates — with fine-grained HTTP semantics.

POST
`/tasks/{id}/complete`

Mark task done

POST `/tasks/{id}/block`

Flag blocked task

Progress & Replanning APIs

Progress APIs

`GET /goals/{id}/progress`

Retrieve real-time progress snapshot for a goal, including completed tasks, blocked items, and milestone status.

Replanning APIs

`POST /goals/{id}/replan`

Trigger AI-driven replanning when circumstances change.

`POST /replans/{id}/accept`

Commit the proposed plan revision.

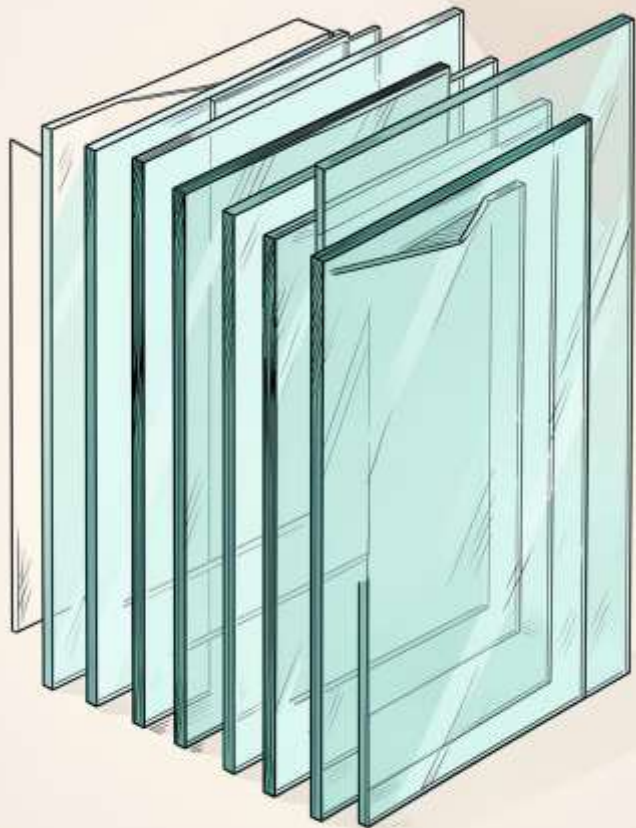
`POST /replans/{id}/reject`

Discard the revision, preserving original plan.

End-to-End API Flow



PathLoom's API layer is **stateless**, with clear separation of concerns between FastAPI (REST interface), LangChain (orchestration), and Azure OpenAI (reasoning). This architecture supports horizontal scaling and future enterprise integration.



Why Not a Single Prompt?

A monolithic prompt forces the LLM to handle goal analysis, task generation, dependency mapping, prioritization, and milestone planning simultaneously – introducing noise, inconsistency, and untraceable failures. PathLoom replaces this with a **staged reasoning pipeline**.

Context Overload

Too many concerns in one context window degrade output quality

Reduced Control

Hard to steer or constrain specific reasoning stages

Inconsistent Outputs

Non-deterministic structure makes downstream parsing fragile

Poor Explainability

Impossible to audit which reasoning step produced a flaw

Prompt Chain Architecture

Six specialized prompts form a sequential pipeline. Each prompt receives a focused, narrow task – enabling modular evaluation, targeted debugging, and reliable structured outputs at every stage.



01

Goal Prompt

Classify type, complexity, duration

02

Task Generation

Break goal into actionable tasks

03

Dependency Detection

Map execution relationships

04

Priority Assignment

Rank tasks by urgency and impact

05

Milestone Planning

Group tasks into checkpoints

06

Workflow Generation

Assemble full executable plan

07

Replanning Prompt

Adapt plan on change or block

Example Output & Key Benefits

Example: Goal Analysis Prompt

Input: *"Build AI SaaS MVP in 30 days."*

Output:

```
{
  "goal_type": "Product Development",
  "complexity": "Medium",
  "estimated_duration": "30 Days",
  "key_workstreams": [
    "Research",
    "Backend",
    "Frontend",
    "Testing"
  ]
}
```

Why This Approach Wins



Modular Reasoning

Each prompt handles one concern — easier to test, tune, and replace independently



Better Evaluation

Stage-level metrics reveal exactly where the pipeline degrades



Easier Debugging

Trace failures to a specific prompt, not a monolithic black box



Reliable Outputs

Structured JSON schemas per stage ensure downstream compatibility



SLIDE 16 — EVALUATION FRAMEWORK

Why Evaluation Matters

LLM systems can generate outputs — but those outputs must be measured. PathLoom evaluates four dimensions of workflow quality to ensure every generated plan is useful, accurate, and adaptive.

Four Pillars of Workflow Quality



Task Quality

Did we generate useful, actionable tasks that move the user toward their goal?



Dependency Quality

Were prerequisite relationships between tasks correctly identified and structured?



Priority Quality

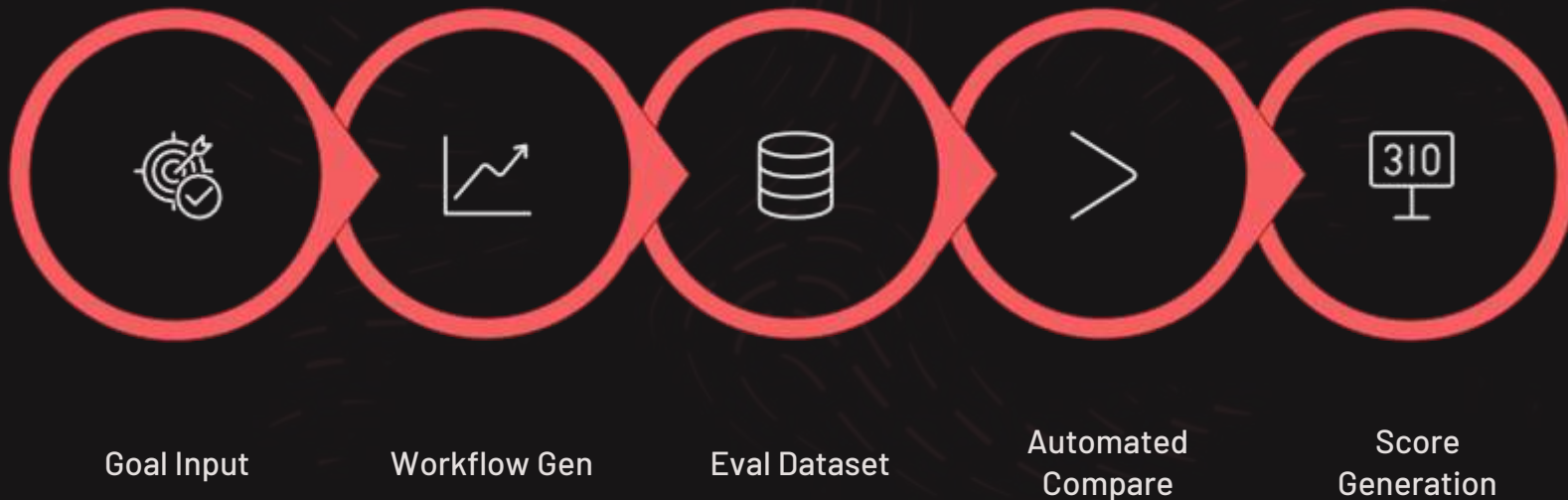
Were critical tasks ranked properly to guide effective execution order?



Replanning Quality

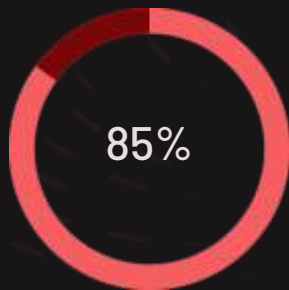
Did workflow adaptation respond logically and coherently to delays or changes?

From Goal to Feedback Loop



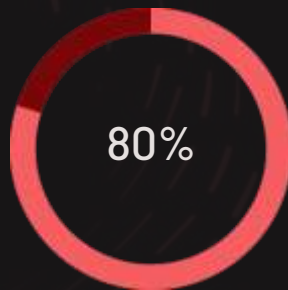
Each goal flows through a structured pipeline — from generation to automated scoring — enabling continuous improvement through measurable, repeatable feedback.

Target Benchmarks



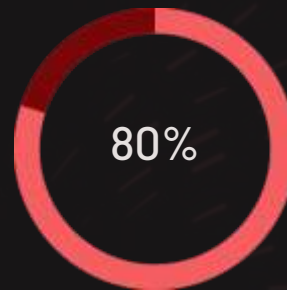
Task Coverage

Percentage of expected tasks successfully generated



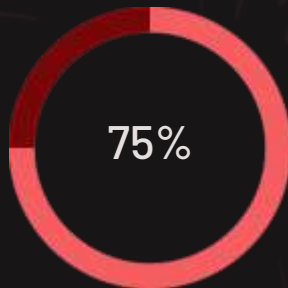
Dependency Accuracy

Correctness of prerequisite relationships detected



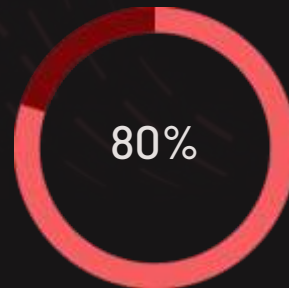
Priority Accuracy

Alignment of assigned priorities with expected ranking



Replanning Consistency

Logical coherence of adaptive workflow adjustments



User Satisfaction

Qualitative rating of workflow usefulness and clarity

Purpose: Realistic Goals, Reliable Evaluation

The golden dataset provides a curated set of realistic user goals with known expected outputs – enabling objective, repeatable evaluation of PathLoom's workflow generation across diverse domains and complexity levels.



DATASET CATEGORIES

Five Goal Archetypes



Student

Learn DSA in 60 Days



Startup

Launch a SaaS MVP



Professional

Prepare for Certification



Developer

Build a Mobile App



Product Manager

Launch a New Feature

Structured Expected Outputs

Schema Overview

Each golden goal includes four expected output arrays used as the ground truth for automated evaluation.

- **expected_milestones** — Key phases of the goal
- **expected_tasks** — Actionable task list
- **expected_dependencies** — Prerequisite relationships
- **expected_priorities** — Critical task rankings

```
{  
  "goal": "Build SaaS MVP",  
  "expected_milestones": [...],  
  "expected_tasks": [...],  
  "expected_dependencies": [...],  
  "expected_priorities": [...]  
}
```

WHY A GOLDEN DATASET?

Three Pillars of Reliable Evaluation

Repeatable Testing

Run the same goals across model versions to track improvements and regressions over time.

Objective Evaluation

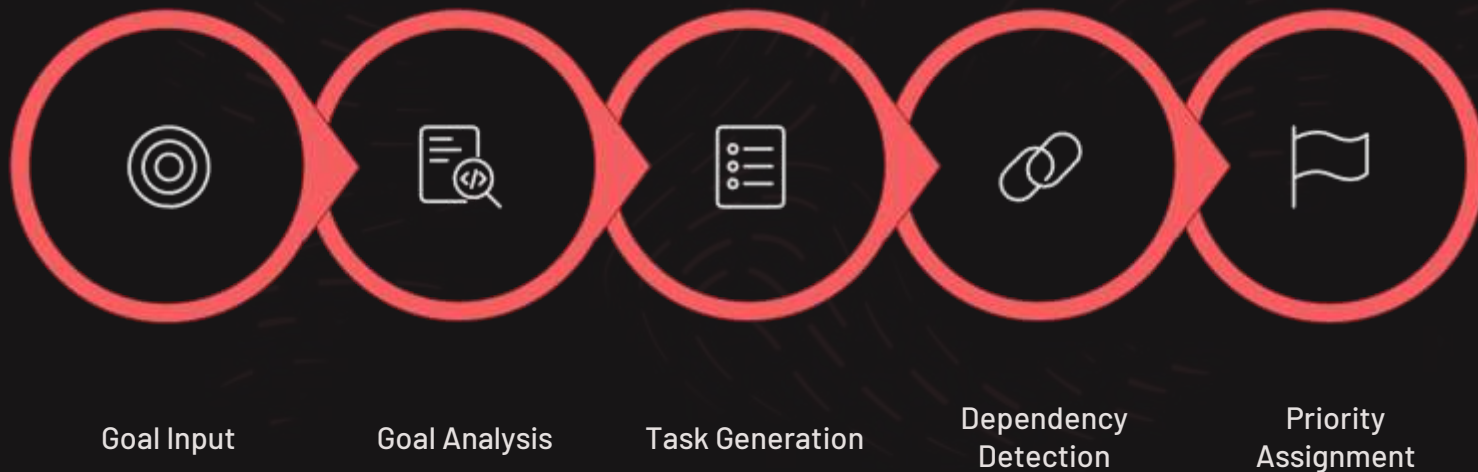
Automated comparison against known outputs removes subjectivity from quality assessment.

Consistent Benchmarks

Shared evaluation standard aligns engineering, product, and stakeholder expectations.

MVP Scope & Workflow

PathLoom's MVP demonstrates six core capabilities — from goal analysis through adaptive replanning — in a single end-to-end workflow.



The MVP covers the full adaptive loop — generating, displaying, tracking, and replanning workflows — without external integrations or enterprise features.

OUT OF SCOPE

What the MVP Does Not Cover



These capabilities are intentionally deferred to keep the MVP focused on validating core LLM-driven workflow generation and adaptation.



Team Collaboration



Jira Integration



Slack Integration



Calendar Sync



Enterprise Permissions

SLIDE 19 – PROTOTYPE WALKTHROUGH

Build an AI SaaS MVP in 30 Days

Watch how PathLoom turns a single user goal into a fully structured, adaptive project plan — powered by intelligent agents working in concert.



STEP 1

User Submits Their Goal



The journey starts with one simple input. The user types their objective into PathLoom – no complex forms, no rigid templates.

 **Example Input:** "Build an AI SaaS MVP in 30 Days"

From this single statement, PathLoom's agent network activates and begins constructing a tailored execution plan.

STEP 2

Goal Analysis Agent Interprets Intent

Goal Type

Product Development

Complexity

Medium

The **Goal Analysis Agent** parses the user's input and classifies it across key dimensions. It identifies this as a **Product Development Goal** with **Medium Complexity** — triggering the appropriate task generation strategy.

This classification ensures the downstream agents tailor their output to the right scope, timeline, and resource profile.

STEP 3

Task Agent Generates the Work Breakdown

Based on the goal classification, the **TaskAgent** automatically generates a structured set of workstreams – covering every phase needed to ship an MVP.



Research

Market validation, competitive landscape, and user needs assessment



Backend

API development, database setup, and server infrastructure



Testing

Unit tests, integration tests, and QA cycles



Architecture

System design, tech stack selection, and data modeling



Frontend

UI components, user flows, and responsive design



Deployment

CI/CD setup, cloud hosting, and production launch

STEP 4

Dependency Graph Is Created



PathLoom maps every task's dependencies, revealing what must happen before what. This graph prevents teams from starting work in the wrong order and surfaces critical path risks early.



Priorities Are Assigned

With the dependency graph in place, PathLoom assigns **priority scores** to each task — balancing urgency, complexity, and downstream impact. High-priority tasks are surfaced first, giving the team a clear starting point and a confidence-backed execution order.

STEP 6

User Adjusts the Plan – Backend Delayed by 5 Days

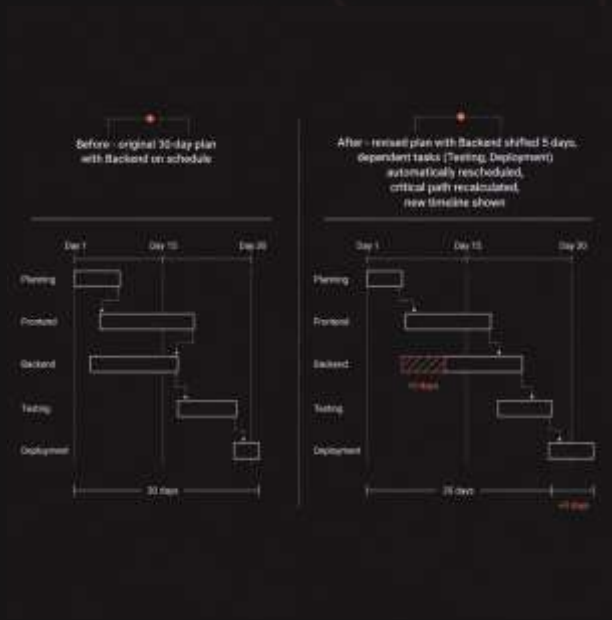


Real projects rarely go exactly as planned. In this scenario, the user signals a **5-day delay on Backend** – perhaps due to a resource constraint or a technical blocker.

i PathLoom accepts this change as a live input, not a one-time plan. The system is designed to adapt, not break.

STEP 7

PathLoom Generates a Revised Workflow



The moment the delay is entered, PathLoom recalculates the entire plan. Dependent tasks shift automatically, the critical path is updated, and the user sees a **revised, conflict-free workflow** in seconds.

The picture can't be displayed.

✓ No Manual Rescheduling

Dependencies propagate changes automatically

The picture can't be displayed.

✓ Always Current

The plan reflects reality, not assumptions



TECHNOLOGY DECISIONS

Technology Selection & Architectural Decisions

Every technology choice was evaluated against alternatives for reasoning quality, integration depth, and development velocity.

Why We Chose These Technologies

The picture can't be displayed. Azure OpenAI Evaluated: Gemini, Claude, Open Source • Strong reasoning & structured outputs • Enterprise adoption & agent workflow integration	The picture can't be displayed. LangChain Evaluated: Semantic Kernel, Custom Orchestration • Agent orchestration & prompt chaining • Rich tool integration, faster MVP
The picture can't be displayed. FastAPI Evaluated: Flask, Express • Native async support, auto API docs • Strong Python ecosystem, faster dev cycle	The picture can't be displayed. PostgreSQL Evaluated: MongoDB • Relational workflow dependencies • Strong consistency for workflow data

Architecture Flow



From MVP to Enterprise Execution Platform

1

Phase 1 – Current MVP

Goal decomposition, dependency mapping, prioritization, progress tracking, adaptive replanning

2

Phase 2 – Collaborative Planning

Shared workspaces, team goals, role-based access, shared dashboards

3

Phase 3 – Tool Integrations

GitHub, Jira, Slack, Calendar Systems

4

Phase 4 – Enterprise Execution

Resource allocation, portfolio planning, delivery risk prediction, cross-team optimization



Long-Term Vision: Task Planner → Workflow Intelligence → Team Intelligence → Enterprise Execution Platform

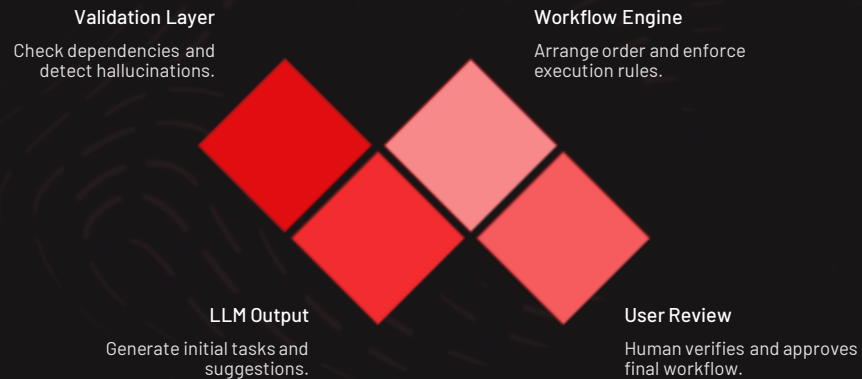


Risks & Validation Strategy

Potential Risks

Risk	Impact	Mitigation
Hallucinated Tasks	Incorrect workflow	Validation layer
Incorrect Dependencies	Wrong execution order	Dependency verification
Poor Prioritization	Reduced usefulness	Scoring framework
Context Loss	Incomplete plans	Workflow memory

Validation Pipeline



Every LLM-generated output is validated before reaching the user, ensuring correctness at each stage of workflow construction.

Explainability Framework

Users must understand **why** tasks were generated, **why** they received their priority, and **why** replanning occurred — transparency drives trust and adoption.

Example Output

Task: Backend API

Priority: High

Reason: Blocks Authentication, Testing, and Deployment.

Why It Matters



Transparency

Users see the reasoning behind every decision



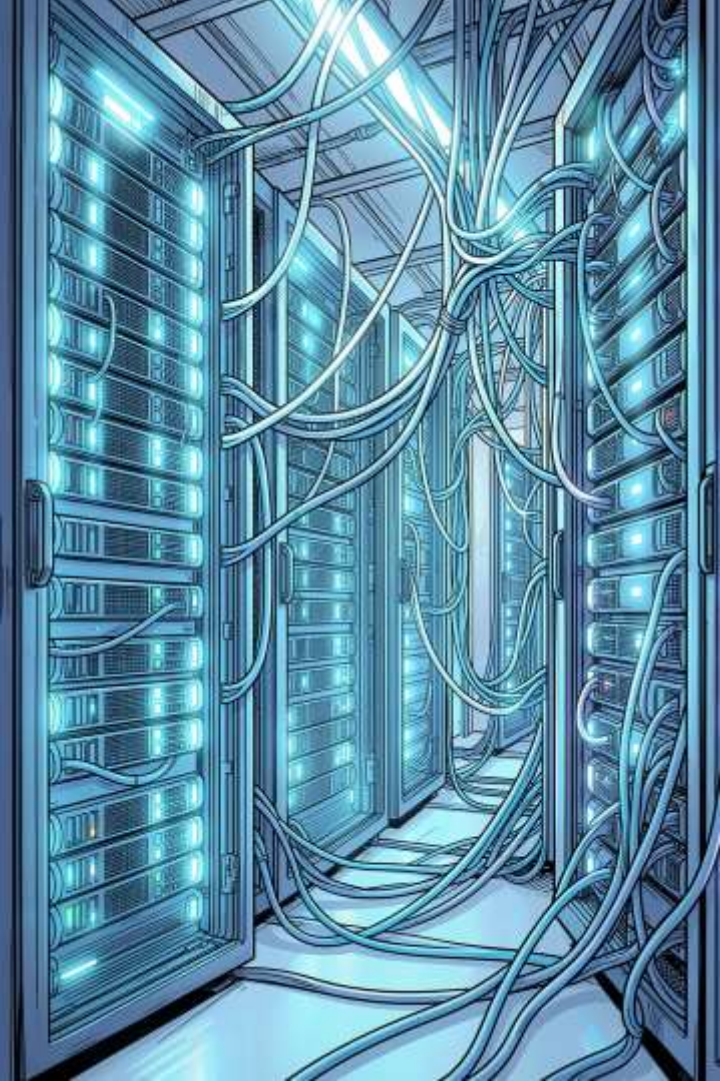
User Trust

Explainable AI reduces skepticism and increases adoption



Better Decisions

Users can override or refine AI suggestions with confidence



SCALABILITY

Scalability & Enterprise Readiness

Current MVP

Single user, single workflow — focused on validating core agentic planning capabilities with minimal infrastructure.

Future Architecture

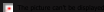
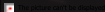
Users → API Gateway → Agent Layer → Workflow Engine → Database → Analytics — designed for horizontal scaling at every tier.

Scalability Features

- Horizontal API scaling
- Queue-based processing
- Modular agent architecture
- Distributed storage support

Security & Governance

What We Store

 The document is locked for editing.	 The document is locked for editing.
Goals, Tasks & Progress Updates	Workflow History

What We Never Store

- Credentials or passwords
- External secrets or API keys

Security Measures



Environment Variables

Secrets isolated from code and runtime



HTTPS & API Key Isolation

Encrypted communication, scoped access



Database Access Control

Role-based permissions, audit-ready design



Designed for future enterprise compliance and governance requirements.

Evaluation Framework

Success Metrics

01

Task Decomposition Accuracy

02

Dependency Accuracy

03

Priority Accuracy

04

Replanning Accuracy

05

Workflow Completion Rate

06

User Satisfaction

Golden Dataset

30 benchmark goals across five user personas, designed to stress-test every agent capability:

Students

Developers

Founders

Professionals

Product Managers

Where PathLoom Stands Alone

Capability	To-Do Apps	AI Chatbots	PM Tools	PathLoom
Task Generation	✗	✓	✗	✓
Dependency Detection	✗	Limited	Manual	✓
Prioritization	Basic	Limited	Manual	✓
Progress Tracking	Basic	✗	✓	✓
Adaptive Replanning	✗	✗	✗	✓



Key Differentiator: Most systems generate plans. **PathLoom adapts plans during execution.**

CONCLUSION

Planning Is Not the Problem. Sustaining Execution Is.

PathLoom transforms vague goals into completed outcomes through adaptive, agentic workflow intelligence.



Thank You

PathLoom— Adaptive Workflow Intelligence Platform

Team CodeScalers

Questions & Discussion